

BOOSTING, ONE ERROR AT A TIME

Gradient Boosting Regression

Start with one rough answer. Then every tree after it predicts the leftover error of the model so far, and we add a small piece of that correction back in.

The Flow We Will Follow

01 Where Gradient Boosting sits, and how it differs from AdaBoost

02 Our dataset: ten flats, and the rent we want to predict

03 Six steps of maths, worked out on all ten rows

04 The learning rate, and why we need it

05 The final function, and the settings you will tune

06 What changes for a classification problem

THE BOOSTING FAMILY

1 AdaBoost

2 Gradient Boosting today

3 XGBoost

The dataset table stays on screen through every calculation slide, so you never have to flip back to see the numbers.

BOTH ARE BOOSTING, BUT NOT THE SAME

AdaBoost Passes Weights. This Passes Errors.

	ADABOOST	GRADIENT BOOSTING
Size of each tree	A stump, depth 1	A proper decision tree, deeper than 1
What each tree learns	The real answer, on a reweighted table	The leftover error of the model so far
Row weights	Yes, wrong rows get heavier	None. Every row stays as it is
Weight on each tree	Its own α , from its error	One small learning rate shared by all
In one sentence	"Pay more attention to these rows."	"Here is how much I am off by. Predict that."

Ten Flats, and the Rent We Want to Predict

#	AREA SQFT	BHK	RENT Y	HOW BIG
1	450	1	12	
2	520	1	13	
3	600	1	14	
4	700	2	18	
5	780	2	19	
6	850	2	20	
7	1000	3	25	
8	1100	3	27	
9	1250	3	33	
10	1400	4	39	

Area and **BHK** are the independent features, our inputs.

Rent is the dependent feature, our output. We write it as y.

The rent is a **continuous number** , not a class, so this is a regression problem.

The bars show the size of each rent at a glance. Watch what happens to them on the next slide.

Six Steps, Then a Loop

- 1 Create a **base model**. One flat answer for every row.
- 2 Compute the **residuals**: the real rent minus what the model said.
- 3 Build a **decision tree** that predicts those residuals from Area and BHK.
- 4 Bring in the **learning rate**, a small number between 0 and 1.
- 5 Update the prediction: old prediction + learning rate $\times$ what the tree said.
- 6 Compute the **new residuals** and go back to step 3 with them.

Every round the residuals shrink a little. That is the whole algorithm.

STEP 1

The Base Model Is Just the Average

#	AREA	BHK	RENT Y	REAL RENT	$\hat{Y}$ BASE	PREDICTION
1	450	1	12		22	
2	520	1	13		22	
3	600	1	14		22	
4	700	2	18		22	
5	780	2	19		22	
6	850	2	20		22	
7	1000	3	25		22	
8	1100	3	27		22	
9	1250	3	33		22	
10	1400	4	39		22	

The base model must not favour any row, so it gives **one answer to everybody** : the average of all the rents.

$$12+13+14+18+19 +20+25+27+33+39 \div 10 = 220 / 10$$

22

Base model, $h_0(x)$. Feed it anything, it says 22.

All the orange bars are now the **same length** . Too high for the small flats, too low for the big ones. Those gaps are the next tree's job.

Residuals: How Far Off We Are

#	AREA	BHK	RENT Y	$\hat{Y}$	R1	too high ◀ 0 ▶ too low
1	450	1	12	22	-10	
2	520	1	13	22	-9	
3	600	1	14	22	-8	
4	700	2	18	22	-4	
5	780	2	19	22	-3	
6	850	2	20	22	-2	
7	1000	3	25	22	+3	
8	1100	3	27	22	+5	
9	1250	3	33	22	+11	
10	1400	4	39	22	+17	

$R1 = y - \hat{y}$

row 1: 12 - 22 = -10 row 2: 13 - 22 = -9 row 10: 39 - 22 = +17

Negative means we guessed **too high** .
Positive means **too low** . The bars show it without reading the digits.

The R1 column adds up to **zero** ,
because the base model was the
average. The overs and unders cancel.

R1 is the column the first decision tree
will be trained to predict.

STEP 3

A Tree That Predicts the Error, Not the Rent

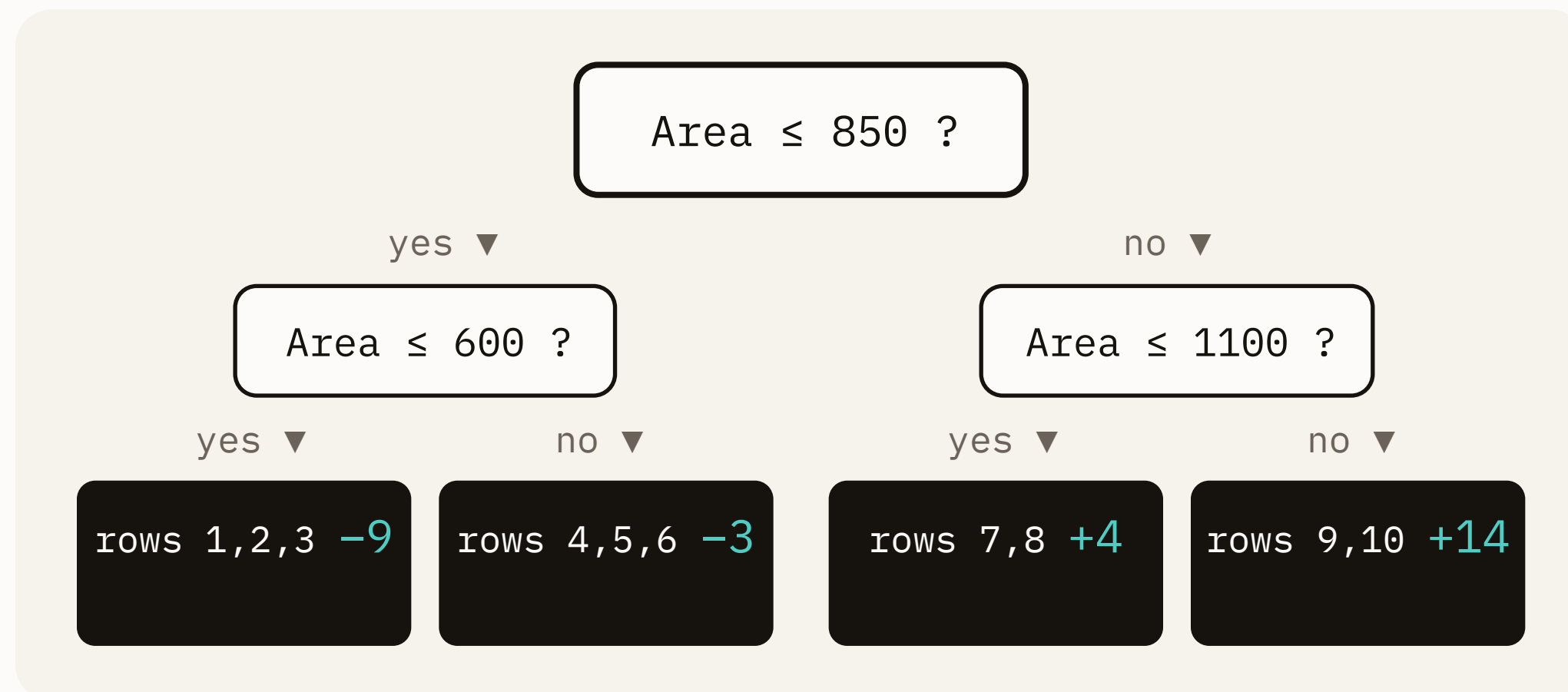
DT 1 trains on $\{x_i, R1\}$ inputs = Area, BHK · output = the residual

The tree is asked one question: **given the area and the bedrooms, how wrong is the model likely to be?**

It is a **proper decision tree**, grown to a normal depth. Not a stump.

Built the usual way for regression: try each feature and threshold, keep the split that reduces **variance** (the same as MSE at a node) the most.

This tree **never sees the rent column**. It only ever sees the errors.



Each Leaf Holds the Average of Its Residuals

Nothing new here. A regression leaf always predicts the **average of the rows that reached it**.

Now DT 1 can answer for **any** flat, not just these ten.

Small flat → "the model is about **9 too high** ." Big flat → "the model is about **14 too low** ."

That is the correction. Next question: how much of it should we actually apply?

LEAF	ROWS	THEIR R1 VALUES	LEAF OUTPUT
Area ≤ 600	1, 2, 3	(−10 − 9 − 8) / 3	−9
601 − 850	4, 5, 6	(−4 − 3 − 2) / 3	−3
851 − 1100	7, 8	(3 + 5) / 2	+4
above 1100	9, 10	(11 + 17) / 2	+14

■ THE TRAP

Adding the Whole Correction Looks Great. It Is Not.

ROW 1, THE TEMPTING WAY

base model $\rightarrow$ 22 DT 1 says $\rightarrow -9$ 22 + (-9)
= 13

real rent = 12 off by 1

Almost perfect after **one** tree. That is not a model that learnt the pattern, it is a model that **memorised the table**.

On a flat it has never seen, it will do badly. This is **overfitting**.

And think what is left for trees 2, 3 and 50 to do. The residuals are already near zero, so **the rest of the model has no job**.

So we do not apply the full correction. We apply a small slice of it, on purpose.

STEP 4

The Learning Rate Takes a Small Slice

THE UPDATE RULE

$$\hat{y}_{\text{new}} = \hat{y}_{\text{old}} + \alpha \times (\text{tree output})$$

α = learning rate, between 0 and 1 here we take $\alpha = 0.1$

ROW 1, DONE PROPERLY

$$= 22 + (0.1)(-9) = 22 - 0.9$$

$$= 21.1$$

Not 13. We moved from 22 to 21.1, while the truth is 12.

One tiny step. For a single tree that looks useless, and it is.

But the next tree looks at what is **still** left over and takes another small step. Then the next. After a hundred trees the steps add up.

Many small careful steps generalise better than one big greedy jump.

The Updated Prediction, All Ten Rows

#	AREA	RENT Y	$\hat{Y}$ BASE	DT 1	$0.1 \times \text{DT1}$	$\hat{Y}_1$	$\hat{y}_1$ vs y
1	450	12	22	-9	$22 - 0.9$	21.1	
2	520	13	22	-9	$22 - 0.9$	21.1	
3	600	14	22	-9	$22 - 0.9$	21.1	
4	700	18	22	-3	$22 - 0.3$	21.7	
5	780	19	22	-3	$22 - 0.3$	21.7	
6	850	20	22	-3	$22 - 0.3$	21.7	
7	1000	25	22	+4	$22 + 0.4$	22.4	
8	1100	27	22	+4	$22 + 0.4$	22.4	
9	1250	33	22	+14	$22 + 1.4$	23.4	
10	1400	39	22	+14	$22 + 1.4$	23.4	

The **DT 1** column is just each row's leaf value, read off the tree.

Rows in the same leaf get the same correction, so they land on the same new prediction.

On the base model slide every orange bar was **identical** . Now the small flats have nudged down and the big ones up.

The shape of the grey column is starting to appear, faintly. That is one tree's worth of progress.

New Residuals, and Every One Is Smaller

#	AREA	RENT Y	$\hat{Y}_1$	R1 WAS		R2 NOW	
1	450	12	21.1	-10		-9.1	
2	520	13	21.1	-9		-8.1	
3	600	14	21.1	-8		-7.1	
4	700	18	21.7	-4		-3.7	
5	780	19	21.7	-3		-2.7	
6	850	20	21.7	-2		-1.7	
7	1000	25	22.4	+3		+2.6	
8	1100	27	22.4	+5		+4.6	
9	1250	33	23.4	+11		+9.6	
10	1400	39	23.4	+17		+15.6	

$R2 = y - \hat{y}_1$

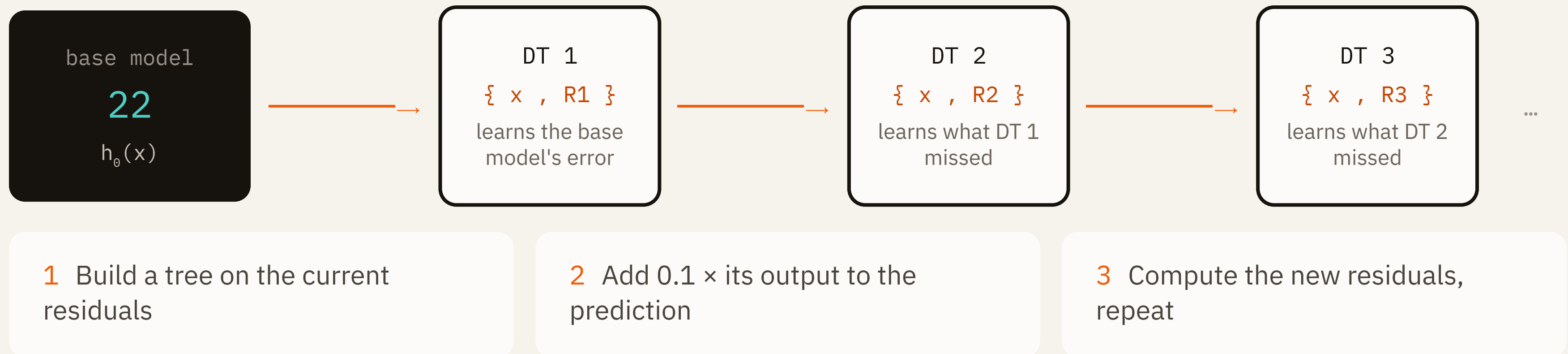
$12 - 21.1 = -9.1$
 $13 - 21.1 = -8.1$
 $39 - 23.4 = +15.6$

Compare the two bar columns.
Every single row moved closer to zero , and nothing got worse.

Only by about one unit, because α is just 0.1. Small steps, on purpose.

R2 is now the target for tree 2. Hand it { x , R2 } and go again.

Each Tree Learns What Is Still Left Over



It stops when we have built as many trees as we asked for. In scikit-learn, `n_estimators = 100` and `learning_rate = 0.1` by default, the same value we just used.

■ THE FINAL FUNCTION

Everything in One Line

$$F(x) = h_0(x) + \alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x) + \dots + \alpha_n h_n(x)$$

$H_0(X)$

The base model. Ours says 22.

$H_1 \dots H_N$

The decision trees, each trained on the leftover error.

$A_1 \dots A_N$

The learning rates. In practice **one shared value**, like 0.1, not one per tree.

$$F(x) = \sum_{i=0}^n \alpha_i h_i(x)$$

Let $\alpha_0 = 1$ for the base model and the whole thing collapses into this. This is the final function of Gradient Boosting, and it covers regression and classification both.

Learning Rate and Number of Trees Pull Against Each Other

A LARGE · 0.9

Big jumps. A good training score in very few trees.

But it overshoots and **overfits**, the trap we saw earlier.

A SMALL · 0.01

Tiny careful steps that generalise well.

But you need a **great many trees**, so training is slow.

THE USUAL STARTING POINT

```
learning_rate = 0.1
```

```
n_estimators = 100
```

Halve the learning rate and you roughly need to double the trees. **Tune them as a pair**, never one alone.

The rest are the ordinary tree settings: `max_depth` (kept small here, around 3 to 5), `min_samples_split` and `min_samples_leaf`.

AND FOR CLASSIFICATION

Same Shape, Two Definitions Change

1 · THE BASE MODEL

average → **log odds**

No longer a plain average of numbers, because the target is a class.

2 · THE RESIDUAL

true label – predicted
probability

Really class 1 but the model says 0.3? The leftover is **0.7**, and the next tree pushes it up.

EVERYTHING ELSE IS THE SAME

Trees on the leftover, each added with a small learning rate. At the end a **sigmoid** turns the total back into a probability.

GradientBoostingRegressor
GradientBoostingClassifier

■ RECAP

Five Things to Carry Forward

- 1 The base model is the **average** of the output. One flat answer for everybody: 22.
- 2 A **residual** is truth minus prediction. That is what the whole algorithm chases.
- 3 Every tree is trained to predict the **residuals**, not the rent. Proper trees, not stumps.
- 4 We add only **$\alpha \times$ the tree's output**. Adding it all at once overfits.
- 5 Recompute the residuals after each tree. They shrink, and the next tree works on what is left.

Next: XGBoost, the same idea made much faster, with regularisation built in.